Deep Learning - COSC2779/2972

Introduction to Deep Learning

A/Prof. Ruwan Tennakoon

RMIT UNIVERSITY

Reference: *Chapter 5: Ian Goodfellow et. al., "Deep Learning", MIT Press, 2016.*

Learning Outcomes

By the end of this lecture you should be able to:

- State a learning problem in terms of **Task (T), Experience (E)** and **Performance measure (P)**.
- Identify the four ingredients of an ML algorithm: dataset, model, cost function, optimisation procedure.
- Diagnose **under-fitting** vs **over-fitting** from training and validation curves.
- Explain why a held-out test set is required, and choose appropriately between **hold-out** and **k-fold cross-validation**.
- Recognise common ways the i.i.d. assumption is violated in practice (data leakage, distribution shift).

Further reading: Goodfellow et al., *Deep Learning* (Ch. 5); Prince, *Understanding Deep Learning* (2023, free online); Bishop & Bishop, *Deep Learning: Foundations and Concepts* (2024).

Recall (§1): deep learning is a specific kind of machine learning. To understand deep learning well, one must have a solid understanding of the basic principles of machine learning. This section recaps those principles in the language we will use throughout the course.

ML: Machine learning is the field of study that gives the ability to learn without being explicitly programmed — Arthur Samuel (1959).

More technically: "A computer program is said to learn:

- Some class of **tasks T**
- From **experience E**, and
- **Performance measure P**

If its performance at tasks in T, as measured by P, improves with experience E". — Tom Mitchell (1998)

The Task can be expressed an **unknown target function**:

$$\mathbf{y} = f(\mathbf{x})$$

- Attributes (features) of the task: $\mathbf{x} \in \mathbb{R}^d$
- Unknown target function: $f(\mathbf{x})$
- Output of the function: $\mathbf{y} \in \mathbb{R}^c$

ML finds a Hypothesis (model), $h(\cdot)$, from hypothesis space $\mathcal{H}$, which approximates the unknown target function.

$$\hat{\mathbf{y}} = h^*(\mathbf{x}) \approx f(\mathbf{x})$$

The (optimal) hypothesis is learnt from the Experience. The hypothesis generalises to predict the output of instances from outside of the Experience.

The Experience is typically a data set, $\mathcal{D}$, of values

$$\mathcal{D} = \left\{ \left(\mathbf{x}^{(i)}, f\left(\mathbf{x}^{(i)} \right) \right) \right\}_{i=1}^{N} \quad \triangleright \text{(Supervised learning)}$$

- Attributes (features) of the task: $\mathbf{x}^{(i)} \in \mathbb{R}^d$
- Output of the function (**Target**): $\mathbf{y}^{(i)} = f\left(\mathbf{x}^{(i)} \right)$

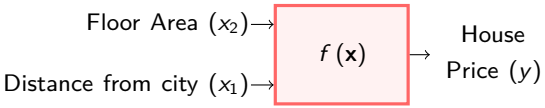
What does success look like? To evaluate the abilities of a machine learning algorithm, we must design a quantitative measure of its performance.

$$\text{We would like to measure:} \quad h^*(\mathbf{x}) \approx f(\mathbf{x})$$

The performance is a numerical measure that determines how well the hypothesis matches the target on a set of examples. We need to distinguish three related quantities:

- **Training error** (also: training loss) — measured on the data used to fit the model. Used *to fit* h.
- **Generalization error** (estimated by **test error**) — measured on a held-out test set drawn from the same distribution. Used *to report* how well h performs.
- **True risk** — the expected loss over the full data-generating distribution p_{data}. Unobservable; we only ever *estimate* it via test error.

Example: Linear Regression



We need to design an algorithm that will improve the **weights**, w, in a way that reduces MSE on the training set, so that the model generalises and achieves low MSE on the (unseen) test set.

One intuitive way of doing this is just to minimize the mean squared error on the training set.

How can we find the w which minimize the performance measure (Mean Squared Error) on train data?

Hypothesis (Model):

$$\hat{y}^{(i)} = h\left(\mathbf{x}^{(i)}\right) = w_0 + w_1 x_1^{(i)} + w_2 x_2^{(i)}$$

Hypothesis space: $\mathcal{H} \in$
All possible combinations of (w_0, w_1, w_2)

Experience:

$$\mathcal{D} = \left\{ \left(\left[x_1^{(i)}, x_2^{(i)} \right], y^{(i)} \right) \right\}_{i=1}^{N}$$

Performance Measure:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left(y^{(i)} - \hat{y}^{(i)} \right)^2$$

performance is measured over the test set.

Common Machine Learning Problems

- **Classification:** Specify which of k categories some input belongs to.
- **Regression:** Predict a numerical value given some input.
- **Anomaly detection:** Observes a set of events and flags some of them as being unusual or atypical.
- **Synthesis and sampling:** Generate new examples that are similar to those in the training data.
- **Denoising:** Predict a clean example x from its corrupted version.
- **Machine translation:** Given an input consisting of a sequence of symbols in some language, convert this into a sequence of symbols in another language.
- ...

Nearly all machine learning algorithms can be described with the following fairly simple recipe:

- Dataset
- Cost function (Objective, loss)
- Model
- Optimization procedure

The first step in solving a ML problem is to **analyse the data and task** to identify the above components.

Design Choices:

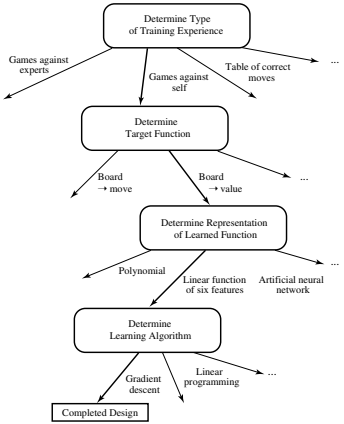


Image: Tom Mitchell, "Machine Learning", 1997.

Generalization

The central challenge in machine learning is that our algorithm must perform well on new, previously unseen inputs (not just those on which our model was trained). The ability to perform well on previously unseen inputs is called generalization.

- Generalization error is related to the true error of a hypothesis (cannot be measured).
- The generalization error of a machine learning model is typically estimated by measuring its performance on a **test set** collected separately from the training set.

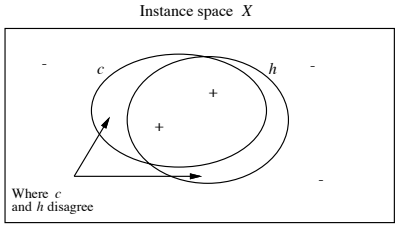


Image: Tom Mitchell, "Machine Learning", 1997.
Here $c := f$ is the unknown target function.

How can we affect performance on the test set when we can observe only the training set?

If the training and the test set are collected arbitrarily, there is indeed little we can do. In practice, the learning algorithm does not actually find the best function, but merely one that significantly reduces the training error.

However, If we are allowed to make some assumptions about how the training and test set are collected, then we can use the field of *statistical learning theory* to obtain some answers.

Assumptions about the data-generating process:

- Each dataset are independent from each other.
- Training set and test set are identically distributed. We call that shared underlying distribution the data-generating distribution (p_{data}).

Example: Data-generating Process

We are interested in **classifying traffic signs in Melbourne, Australia**. To train an ML model, we use the following process: randomly pick a traffic sign (`signID`) and image it three times (`instanceID`) from slightly different angles, crop the traffic sign and save it as "`signID_instanceID.jpg`".

Discuss the following three scenarios:

1. **Near-duplicate leakage:** all images with `instanceID` $= 1$ are the test set; the rest are the training set.
2. **Distribution shift:** all our images are used for training; the *German* traffic-sign dataset is used as the test set.
3. **Correct split:** partition by `signID` — a sign is in the training set or the test set, but never both.

Which scenarios violate the i.i.d. assumption, and how?

Under-fitting and Over-fitting

The factors determining how well a machine learning algorithm will perform are its ability to

- Make the training error small.
- Make the gap between training and test error small (generalization)

Two main challenges in machine learning:

- **Under-fitting:** Both training and test error is large. The model does not have enough capacity to capture the target function.
- **Over-fitting:** Test error is large but the training error is small (large gap). Learning vs. memorizing.

$*$The expected test error is greater than or equal to the expected value of training error.

Under-fitting and Over-fitting

We can control whether a model is more likely to over-fit or under-fit by altering its **capacity**.

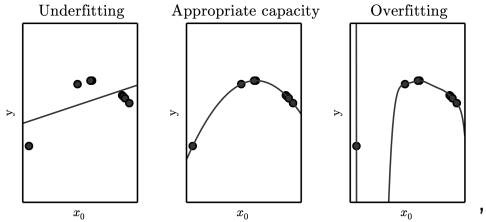


Image: Goodfellow 2016

One way to control the capacity of a learning algorithm is by choosing its hypothesis space $\mathcal{H}$. Linear $\leftrightarrow$ polynomial.

Machine learning algorithms will generally perform best when their capacity is appropriate for the true complexity of the task they need to perform and the amount of training data they are provided with.

Generalization Gap

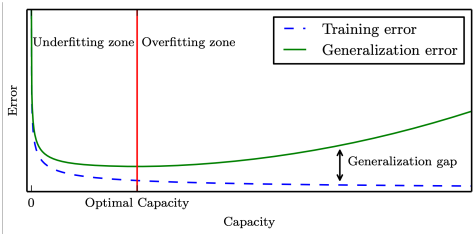


Image: Goodfellow, 2016.

Simpler functions are more likely to generalize, but a sufficiently complex hypothesis is needed to achieve low training error.

∗ It is possible for the model to have optimal capacity and yet still have a large gap between training and generalization errors. In this situation, we may be able to reduce this gap by gathering more training examples.

Occam's razor: Among competing hypotheses that explain known observations equally well, we should choose the "simplest" one.

Statistical learning theory provides various means of quantifying model capacity. e.g. Vapnik-Chervonenkis dimension (VC-dimension)

- Discrepancy between training error and generalization error is bounded from above by a quantity that grows as the model capacity grows but shrinks as the number of training examples increases.
- These bounds are rarely used in practice when working with deep learning algorithms.
- This is because the bounds are often quite loose and it can be quite difficult to determine the capacity of deep learning algorithms.

Modern view: over-parameterisation and double descent

The classical U-curve predicts that test error rises once model capacity exceeds the "sweet spot". **Modern deep networks contradict this.**

Empirically (Belkin et al., 2019; Nakkiran et al., 2021):

- Test error first follows the classical U-curve.
- At the **interpolation threshold** (model can fit training data exactly), test error *spikes*.
- Beyond that, in the **over-parameterised** regime, test error *decreases again* — "double descent".

State-of-the-art deep nets operate in this over-parameterised regime. Why they generalise is an open question: candidate explanations include the *implicit bias of SGD* towards low-norm / flat-minimum solutions, the *neural tangent kernel*, and benign overfitting.

Takeaways for this course:

- The classical U-curve is still a useful diagnostic for small models and tells the right story for over-fitting in the under-parameterised regime.
- But "bigger model $\Rightarrow$ worse generalisation" is **not** a reliable rule for deep networks.
- Regularisation, optimisation choice, and architecture matter more than raw parameter count.

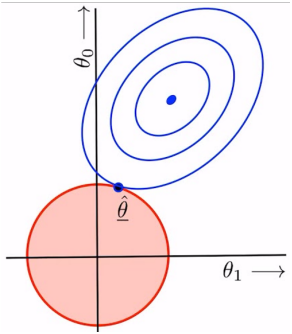
Another method to control whether a model is more likely to over-fit or under-fit is regularization.

Regularization is any modification made to a learning algorithm that is intended to reduce its generalization error but not its training error.

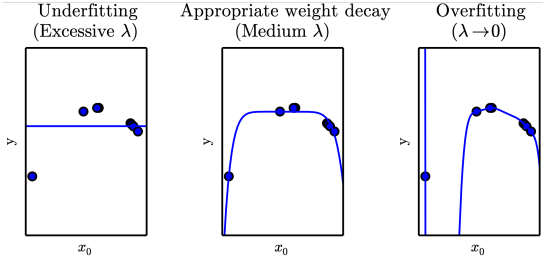
We do so by building a set of preferences (biases) into the learning algorithm. e.g. Preference one solution over another in its hypothesis space.

For example, we can modify the training criterion for linear regression to include weight decay.

$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y - \hat{y})^2 + \lambda w^\top w$



Most machine learning algorithms have hyperparameters - settings that we can use to control the algorithm's behavior.



It is not appropriate to learn hyperparameters on the training set. e.g. If hyperparameters that control model capacity are learned on the training set, it would always choose the maximum possible model capacity, resulting in over-fitting.

$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} (y - \hat{y})^2 + \lambda w^\top w$

To solve this problem, we need a **validation set**.

Dividing the dataset into a fixed training set and a fixed test set (Hold-out validation) can be problematic if it results in the test set being small.

Cross validation provides an alternative to use all the examples in the estimation of the mean test error, at the price of increased computational cost.

In CV training and testing computation are repeated on different randomly chosen subsets or splits of the original dataset.

In k-fold cross-validation procedure, partition the dataset is generated by splitting it into k non-overlapping subsets.

Hold-out: Dividing the dataset into a fixed training set and a fixed test set.

K-fold-CV: partition the dataset is generated by splitting it into k non-overlapping subsets.

Independent (i.i.d) test set to make the final evaluation.

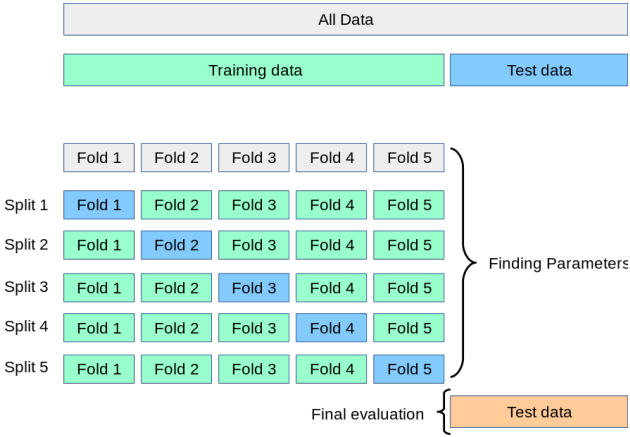


Image: scikit-learn

Review Questions

- Is the performance evaluated over training examples? why?
- What are the key ingredients of a general ML recipe?
- What is generalization-gap?
- If a model shows low train error and high test error is it over-fitting or under-fitting?
- What are the methods that can be used to control whether a model is more likely to over-fit or under-fit?
- Can we use train error to identify the best value for the regularization parameter? Why?
- When will you chose hold-out validation over cross-validation.